

# HDX-INT-DEV: DX Integration for Developers

Study Guide (mapped to SAP Commerce / Hybris)

## Contents

|                                                                                               |          |
|-----------------------------------------------------------------------------------------------|----------|
| <b>Course 3 — HDX-INT-DEV: DX Integration for Developers</b>                                  | <b>1</b> |
| THEORY GUIDE (mapped to SAP Commerce / Hybris)                                                | 1        |
| 1. The three integration <i>patterns</i> (illustrated with HCL Leap, but reusable everywhere) | 1        |
| 2. HCL Commerce integration — the one most like your day job                                  | 2        |
| 3. HCL Volt MX integration — lighter touch, but same shape                                    | 4        |
| 4. Generic external access to DX Assets & Content — this is your OCC toolkit                  | 4        |
| Summary cheat-table                                                                           | 5        |
| <b>Course 3 — HDX-INT-DEV: DX Integration for Developers</b>                                  | <b>7</b> |
| PRACTICAL CHEAT SHEET                                                                         | 7        |
| A. HCL Leap integration (3 patterns)                                                          | 7        |
| B. HCL Commerce integration                                                                   | 8        |
| C. HCL Volt MX integration (lighter-touch reference)                                          | 10       |
| D. Generic external Asset & Content access (URL pattern reference)                            | 10       |

## Course 3 — HDX-INT-DEV: DX Integration for Developers

### THEORY GUIDE (mapped to SAP Commerce / Hybris)

*Source docs: DX Integration with HCL Leap · DX Integration with HCL Commerce · DX Integration with HCL Volt MX · Integrate DX Assets and Content*

This course is about DX as an **integration hub** — pulling in external systems (a forms/low-code engine, a commerce engine, a mobile app platform) and exposing its own content/assets *out* to other systems. Since you're an SAP Commerce backend developer, the most directly relevant sub-topic is **HCL Commerce integration** — note that **HCL Commerce is a completely different product from SAP Commerce/Hybris** (formerly WebSphere Commerce, an unrelated IBM/HCL lineage) — but its architecture (catalogs, categories, Solr search, REST APIs, a management console) will feel very familiar.

---

### 1. The three integration *patterns* (illustrated with HCL Leap, but reusable everywhere)

HCL Leap is a low-code forms/app engine (think: an external microservice with its own UI, similar in spirit to a 3rd-party review widget like Bazaarvoice, or an internal forms tool). DX integrates it three different ways — and this progression is the conceptual skeleton for every integration in this course, including Commerce:

| Pattern                                   | What it does                                                                                                                                                                                                                                              | Hybris/backend analog                                                                                                                                                |
|-------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>1. Embedded App (SDK widget)</b>       | DX content stores config (appId, formId, design choice); a Presentation Template loads the external system's <b>JS SDK</b> (leap.js) and calls its client-side API with that config                                                                       | Dropping a 3rd-party JS SDK (Bazaarvoice/Yotpo reviews) into a CMS component, with the component's custom fields feeding the SDK's init config                       |
| <b>2. Digital Data Connector (DDC)</b>    | A <b>Data Source</b> component makes a server-side/declarative <b>REST call</b> to the external system (filtered by a parameter, e.g. product ID); a <b>List Appearance</b> (Personalization "Select Pluggable Resources" rule) renders the returned list | A Populator/Facade that calls an external REST API to hydrate a component — except DX does it <i>declaratively</i> via a Personalization rule instead of custom Java |
| <b>3. Script Application (custom SPA)</b> | Full custom HTML/CSS/JS (often a small Vue/React app) calling the external system's REST API directly, no black-box SDK                                                                                                                                   | Hand-rolling a custom Spartacus component / micro-frontend against a partner's REST API — maximum control, maximum code                                              |

**Progression = control vs. effort**, same trade-off you already navigate when choosing between "use vendor's SDK," "call vendor REST API from your backend," or "build a fully custom integration."

This exact 3-pattern menu reappears for **HCL Commerce**: Commerce integration in DX is built almost entirely with **pattern 2 (DDC)** for lists (category/product browsing) and simple custom HTML/JS (closer to pattern 3) for cart/wishlist/checkout widgets — no embedded SDK widget pattern is used for Commerce in this lab.

## 2. HCL Commerce integration — the one most like your day job

HCL Commerce (not SAP Commerce) still gives you a near 1:1 vocabulary match:

| HCL Commerce / DX integration concept                                                                                                             | SAP Commerce / Hybris equivalent                                                                                                                 |
|---------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Store</b> (e.g. AuroraESite, Emerald, Sapphire — identified by numeric Store ID)                                                               | <b>BaseStore</b> (e.g. electronics, apparel-uk)                                                                                                  |
| <b>Base Category ID</b> (scopes which catalog subtree a page shows)                                                                               | A <b>Category</b> used as the root for a catalog-driven CMS component (e.g. root category for a "shop by department" nav)                        |
| <b>Global Config content</b> (Commerce Host URL, Store ID, Base Category ID, Shop Page name) — a single content item holding integration settings | A <b>BaseStore/Website configuration</b> — central settings that many components read from, similar in spirit to a site's BaseSiteService config |
| <b>Management Center</b> (wcsadmin/wcsadmin)                                                                                                      | <b>Backoffice</b> — admin cockpit to browse stores/catalogs                                                                                      |

| HCL Commerce / DX integration concept                                                                                            | SAP Commerce / Hybris equivalent                                                                                                                                                                                             |
|----------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Commerce REST Catalog APIs</b><br>(.../catalog_transaction...) via Solr search                                                | Hybris <b>OCC catalog/search endpoints</b> backed by <b>SolrFacetSearchConfig</b> — same “search-index-backed product/category retrieval” pattern                                                                            |
| <b>Category Browsing / Product Browsing page components</b> (DDC-driven lists)                                                   | A <b>CMS component backed by a Solr-indexed category/product query</b> , e.g. a “browse by category” widget                                                                                                                  |
| <b>Product Filter</b> (facet-based filtering)                                                                                    | <b>Solr facets</b> on a category page (ProductSearchFacetValueData)                                                                                                                                                          |
| <b>eSpot</b> (“Get eSpot Details API,” seen in Postman lab)                                                                      | A <b>Content Slot</b> — a named placeholder resolved to marketing content at render time; DX’s “Marketing Content” library items map onto eSpot content the same way a CMSParagraphComponent maps onto a Hybris Content Slot |
| <b>Personalization “Configure Spot” + Hide Spot with segment</b> (e.g. show Product Details only when isCommerceProductSelected) | Hybris <b>Personalization / Smart Merchandising rule</b> conditionally rendering a component based on session/cart state                                                                                                     |
| <b>theme-commerce.html injected via com.ibm.portal.theme.inherited.template.file.name</b>                                        | Similar to injecting <b>storefront-wide JS</b> (cart file, name state) into the Accelerator’s global header/footer JSPs                                                                                                      |
| <b>Single Sign-On note:</b> “in this demo, DX and Commerce don’t share a user directory — in production they should”             | Same caution you’d give in Hybris: SSO between a CMS/portal layer and the commerce backend needs a shared IdP (LDAP/OAuth) in production, never assume it “just works” out of a demo                                         |

### The Commerce DDC chain, concretely (this is the part to memorize, it’s the whole integration pattern):

Content (config: which category/product to browse)  
→ Data Source component (calls Commerce Catalog REST API, Solr-backed)  
→ List Appearance / List Presentation (renders results: header/result/footer design, using DDC “AttributeResource” tags – same idea as looping over a FlexibleSearch result set in a JSP with <c:forEach>)

Everything else (Product Details, Cart, Mini Cart, Wishlist, Category Breadcrumbs) is **plain HTML/CSS/JS page components** that call Commerce’s JS/REST directly — not DDC-driven, because they don’t need a *list*, just live state.

**Customizing an existing integration = copy-and-modify, not rebuild:** the lab’s pattern for adding a “small description” product sticker variant is: open the existing **List Appearance**, **Save As** under a new name, tweak the HTML/CSS in the **Result Design**, then re-point your content’s **List Appearance** field at the new one, and finally **Move** it into the same Site Area as the other reusable Commerce page components so business users can find it. This is the DX equivalent of copying an existing Accelerator JSP fragment/CSS class rather than writing a brand-new component from scratch.

### 3. HCL Volt MX integration — lighter touch, but same shape

Volt MX is a **low-code mobile/web app builder** (think: an internal rapid-app-dev platform, roughly like Mendix or a proprietary MDK) with a back-end integration layer called **Foundry**.

| Volt MX concept                                                                                          | Rough analog                                                                                                                                                                                             |
|----------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Volt MX Foundry</b>                                                                                   | An <b>integration/orchestration layer</b> — closest analog is an iPaaS or an internal API gateway that fronts multiple backend services (including DX) for consumption by apps                           |
| DX Assets/Content consumed <b>via REST/GraphQL through Foundry</b> into a native mobile or web app       | Consuming Hybris <b>OCC</b> from a native mobile app or SPA — DX plays the role of “headless content/asset backend,” Volt MX plays the role of “the app shell consuming it”                              |
| <b>Volt MX Iris</b> — wraps an existing DX <i>site</i> (not just content) into a native mobile app shell | Conceptually like wrapping a responsive Accelerator storefront in a WebView-based hybrid app — DX supplies the full rendered site, Iris just native-wraps it                                             |
| Foundry integrating <b>external data sources into DX</b>                                                 | The reverse direction — Foundry acting as a connector DX can pull <i>other</i> systems’ data through, similar to how you might front an ERP/OMS behind an integration layer before exposing it to Hybris |

This is the least backend-Java-relevant part of the course — treat it as “know it exists, know the shape,” not something to memorize deeply unless you’re specifically asked to integrate a Volt MX app.

### 4. Generic external access to DX Assets & Content — this is your OCC toolkit

This module isn’t about a named 3rd-party product — it’s the **general-purpose API surface** any external system (custom app, another CMS, a script) would use to pull DX data. You already mapped most of this in Course 2’s Experience API section; here it’s applied specifically to *external* (non-DX) consumers.

| Access pattern                              | DX mechanism                                                                                                                   | Hybris equivalent                                                       |
|---------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------|
| Direct binary download link                 | DAM REST getRendition: /dx/api/dam/v1/collections/{collectionId}/items/{assetId}/renditions                                    | mediaService.getMedia(product, {code}, {assetId})/renditions            |
| Friendly addressing by name instead of UUID | Collection <b>Unique name</b> + asset <b>file name</b> + named renditions (Original/Desktop/Smartphone/Tablet) in the URL path | Hybris’s friendly /medias/{code}?context=... URLs vs. raw PK-based URLs |
| Fully custom short URL for one asset        | <b>Custom URL</b> field on the asset (/dx/api/dam/custom/{name})                                                               | A vanity/friendly media URL override                                    |

| Access pattern                                                | DX mechanism                                                                                                | Hybris equivalent                                                                       |
|---------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| Full rendered site/page                                       | Top-level portal URL<br>(/wps/portal/{site}[//{page}])                                                      | Just hitting the Accelerator storefront URL directly — no API involved                  |
| One content item, using DX theme                              | /wps/wcm/connect/{library}/N/A (theme doesn't have a bare “render this CMS item standalone” URL by default) | Site-Header/Content have a bare “render this CMS item standalone” URL by default        |
| One content item, specific presentation, no theme chrome      | ...?presentationtemplate={Closest / {template}}                                                             | Closest / {template} single JSP fragment directly, bypassing the full page layout       |
| A <b>component</b> (menu, list, personalization) in isolation | ...?srv=cmpnt&source=library                                                                                | Rendering as in DX CMS component's JSP standalone, outside its normal page/slot context |
| Read one content item, headless (JSON)                        | WCM REST v1:<br>/dx/api/core/v1/{dxrest mydxrest}/mycontent/authenticated                                   | OCC<br>dxrest / mydxrest authenticated GET                                              |
| Search content by type                                        | /dx/api/core/v1/{access_type mydxrest}/wcm/standalone/SEARCH                                                | File with REST endpoint<br>{itemtype}=?type exposed via OCC                             |
| Create/update content, headless                               | WCM REST v2:<br>/wps/mycontenthandler/wcmrest/v2/contents[//{id}]<br>(GET/POST/PUT)                         | OCC/Backoffice write endpoints                                                          |

**Same governance rule as everywhere else in DX:** every one of these endpoints is still gated by the **Virtual Resource security model from Course 1** (e.g. WCM REST SERVICE needing Editor access for anonymous/authenticated users before external calls work) — API access in DX is never a separate auth system bolted on the side, it's the *same* permission model you already learned.

## Summary cheat-table

| If you know this in Hybris...                                                            | ...think of this in DX Integration                                                            |
|------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| 3rd-party SDK widget in a CMS component<br>Populator/Facade calling an external REST API | Leap “Embedded Application” pattern<br>Digital Data Connector (Data Source + List Appearance) |
| Custom Spartacus micro-frontend<br>BaseStore                                             | Script Application calling REST directly<br>Commerce “Store” (Store ID)                       |
| Category root for a catalog widget<br>SolrFacetSearchConfig-backed search                | “Base Category ID”<br>Commerce Catalog REST API (Solr-backed)                                 |
| Content Slot                                                                             | eSpot / Marketing Content                                                                     |
| Smart Merchandising / Personalization rule                                               | “Configure Spot” + segment-based Hide Spot                                                    |
| iPaaS / API gateway fronting multiple backends                                           | Volt MX Foundry                                                                               |
| Hybrid WebView app wrapping a storefront<br>mediaService.getMedia(format)                | Volt MX Iris wrapping a DX site<br>DAM rendition REST call                                    |
| OCC anonymous/customer GET                                                               | WCM REST v1 (dxrest / mydxrest)                                                               |

| If you know this in Hybris... | ...think of this in DX Integration |
|-------------------------------|------------------------------------|
| OCC/Backoffice write endpoint | WCM REST v2                        |

-e

# Course 3 — HDX-INT-DEV: DX Integration for Developers

## PRACTICAL CHEAT SHEET

---

### A. HCL Leap integration (3 patterns)

#### A1. Pattern 1 — Embedded Application (SDK widget)

1. Find the content built on **Leap Application - CT** → **Open Edit Form** → note Application UUID + Form ID under **Configuration**, and the **Design** reference under **Presentation Style**.
2. Trace the rendering chain: Leap Application - CT → **Default Presentation Template** = Embedded Leap Application - PT → calls [Component name="leap/general/leap application css - hc"] for styling, then renders the content's selected **Design** component.
3. That Design component (e.g. Leap Application - Simple - HC) in turn calls: [Component name="leap/general/embed leap application - hc"] → [Component name="leap/general/embed leap application basic - hc"]
4. Inside that base component: loads /apps/api/leap.js, then:

```
Leap.onReady = function() {  
  let launchParams = {  
    appId: '[Element ... key="appId"]',  
    formId: '[Element ... key="formId"]',  
    locale: '[Element ... key="locale"]',  
    targetId: '[Plugin:ScriptPortletNamespace]leapDiv',  
    prefSecMode: prefSecMode  
  };  
  // Leap.launch(...) call using launchParams  
};
```

5. **Note:** embedded Leap apps do **not** render in Edit Mode (shows a placeholder message instead) — expected, not a bug.
6. Business-user config UI: **leap/general/leap application selector - hc** component lets users browse published Leap apps and write the selection back via REST — no code needed to swap which form is embedded.
7. To see all usages: **Authoring Templates** → Leap Application - CT → **More** → **View References**.

#### A2. Pattern 2 — Digital Data Connector (DDC list)

1. Find content built on **Leap DDC - CT** → **Open Edit Form** → check **Data Source** and **List Appearance** component references.
2. **Data Source** (e.g. Product-ID-specific Approved Leap submissions - HC): HTML Markup calls Leap's **List Data REST API**, filtered by a render parameter (e.g. F\_ProductID = <renderParam 'filters'>), returns only Approved submissions.
3. **List Appearance** (e.g. Product Reviews - LP): a **DDC Personalization Component** — "Select Action" rule type → **Select Pluggable Resources**, wired to the Data Source above. Its *List Presentation Markup* uses Leap's **Data Access REST API** to paginate/render results.

4. To see more examples: **Authoring Templates** → **Leap DDC** - **CT** → **More** → **View References**.

### A3. Pattern 3 — Script Application (custom SPA)

1. Find examples: **Web Content** → **Leap library** → **Content** → **Leap Applications** → **Product Review Components** (e.g. *Product Review Manager*, *Product Rating Summary*).
  2. Structure = standard Script Application: **HTML tab** (Vue.js CDN includes + markup), **js/app.js** (logic), **css/style.css** (styling, often left empty — theme provides base styles).
  3. It's a full custom SPA calling Leap's REST API directly — no SDK, no DDC — maximum control.
- 

## B. HCL Commerce integration

### B1. Set up Shop + Checkout pages

1. Log in as ghayes (or wpsadmin) → **Web Content** → **Commerce library** → **Content** → **Global Config** → **Global Config** → **Edit**.
2. Set: **Commerce Host URL**, **Store ID** (e.g. 1=AuroraESite, 11=Emerald, 12=Sapphire...), **Base Category ID** (optional — scopes catalog subtree), **Shop Page** name (the page other components link back to).
3. Find real Store IDs in **Management Center** (wcsadmin/wcs1admin) → **Manage Stores** → **Store Management** → **Stores** → double-click a store to see its numeric ID.
4. Build the Shop page from ready-made page components: **Category Browsing**, **Product Filter**, **Product Browsing**, **Product Details**, **Basket**.
5. Use **Personalization** → **Configure Spot** → **Hide Spot** with segments under isCommerceProductSelected to swap views:
  - Hide **Product Browsing** when commerceProductsIsSelected
  - Hide **Product Details** / **Basket** when commerceProductsIsNotSelected
6. Build a separate **Checkout** page (friendly URL = checkout, matching what **Commerce** → **Components** → **List Appearances** → **Shop** → **Mini Cart** - HC expects) with **Cart** + **My Wishlist** components.
7. Confirm the **Shop Page** name is set correctly back in **Global Config**.

### B2. Trace the existing integration (read-only exploration)

1. **Web Content** → **Authoring** → **Library Explorer** → **Commerce** → **Content** — top-level folders: Global Config, Commerce (page components), Theme, Electronic Forms (Leap app reference).
2. Open **Category Browsing** content → **Data Source** = Category Browsing Handler — HC (calls Commerce **Catalog REST API**, Solr-backed, branches on whether Base Category ID is set) → **List Appearance** = Category Browsing Carousel — LP (DDC pluggable resources; result design uses AttributeResource tags + a swiper.js carousel UI).
3. Simpler components (**Product Filter**, **Product Details**, **Cart**, **Mini Cart**, **Wishlist**, **Category Breadcrumbs**) = plain HTML/CSS/JS List Presentations calling Commerce JS/REST directly — no DDC.
4. **Theme integration** (SSO + navigation injection): Content → Theme items are wired into the theme via dynamic content spots, e.g. in theme-commerce.html:

```

<a rel='dynamic-content' href='dyn-
↳ cs:id:st_wcm_content?path=commerce/theme/customBannerIcons.html'></a>
<a rel='dynamic-content' href='dyn-
↳ cs:id:st_wcm_content?path=commerce/theme/loginButton.html'></a>
<a rel='dynamic-content' href='dyn-
↳ cs:id:st_wcm_content?path=commerce/theme/injectCommerceNavigation.html'
↳ ></a>

```

selected via page parameter `com.ibm.portal.theme.inherited.template.file.name.html`.

5. Structural reference: **Commerce → Authoring Templates** (Global Config CT, DDC CT), **Commerce → Presentation Templates**, **Commerce → Components** (Data Sources / List Appearances / General reusable JS components).

### B3. Customize: clone a List Appearance for a new page component variant

(worked example: shrink description text on a “Product Sticker” component) 1. Add existing **Promo - Product Sticker** page component to a test page, configure a few product IDs, save. 2. Open its content → **Open Edit Form** → note the **productIdList** element and current **List Appearance** reference. 3. Open that List Appearance (e.g. Promo – Product Stickers – LP) → **Edit** → **Save and Close menu** → **Save As** → give it a new name, e.g. Promo – Small Description Product Stickers – LP. 4. Edit the **List Presentation Markup**: add a CSS class in the header — `css .fs-08 { font-size: 0.8rem; }` — then append `fs-08` to the class attribute on the relevant result-design `<a>` tags. 5. Save and close. Update your original content: rename it, and re-point its **List Appearance** field to the new one → **Save and Read**. 6. **Move** the content into the same Site Area as the other reusable Commerce page components (More → Move → target Commerce/Commerce site area, placement Before/After sibling) so it appears in the **Page Components → Commerce** toolbar for business users. 7. Optionally update its preview-image + description so users can identify the new variant.

### B4. Explore DX content feeding a Commerce-side site (reverse direction)

1. **Digital Assets** — note existing Commerce asset **Collections** (per-store marketing imagery).
2. **Web Content** — Commerce Marketing libraries (English/Spanish) with dedicated **Content Templates** (e.g. Marketing Content) — inspect via **Edit** → **Default Content** to see field types/required rules.
3. Example: a store’s Hero banner content (e.g. `OrderManagementContentSapphire`) — content built on a Marketing Content Template, consumed by the Commerce-side storefront.

### B5. Postman: call DX APIs feeding a Commerce React storefront

1. Import environment (HCL Commerce SoFy CMS-DAM Demo - `Environment.json`) → update hostname var to your sandbox.
2. Import collection (HCL Commerce SoFy CMS-DAM Demo – `API Collection.json`).
3. Run **Get eSpot Details API** → verify eSpot Name: Home Hero + a `contentUrl` in the response.
4. Run **Get Marketing Content Details API** → verify values match what you see authored in DX.

## C. HCL Volt MX integration (lighter-touch reference)

High-level shape only — treat as “know where to look,” not a deep-dive:

| Task                                                       | Where                                                                                     |
|------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| Use DX <b>Digital Assets</b> in a Volt MX mobile app       | Foundry REST config pointing at DAM API endpoints (see Course 2, section D4 URL patterns) |
| Use DX <b>Content</b> in a Volt MX web app                 | Foundry REST/GraphQL config pointing at WCM REST endpoints                                |
| Integrate external data sources <b>into DX</b> via Foundry | Foundry acts as the connector/orchestration layer DX calls out to                         |
| Integrate a Volt MX <b>Foundry web app</b> into DX         | Embed similarly to a Script Application / iframe-style component                          |
| Turn a DX <b>site</b> into a native app                    | <b>Volt MX Iris</b> wraps the rendered DX site in a native shell                          |

## D. Generic external Asset & Content access (URL pattern reference)

### Assets (DAM)

# Direct rendition download (binary)

GET /dx/api/dam/v1/collections/{collectionId}/items/{assetId}/renditions/{renditionId}?binary

# Same, using friendly collection unique-name + asset file-name instead of UUIDs

GET /dx/api/dam/v1/collections/{collectionUniqueName}/items/{fileName}/renditions/{renditionId}

# Named rendition shortcuts: Original | Desktop | Smartphone | Tablet

GET /dx/api/dam/v1/collections/{collectionUniqueName}/items/{fileName}/renditions/{renditionName}

# Fully custom short URL (set on the asset's "Custom URL" field)

GET /dx/api/dam/custom/{customName}

### Content — direct rendered access (uses DX theme/presentation)

# Full site

GET /wps/portal/{siteName}

# One page

GET /wps/portal/{siteName}/{friendlyPageURL}

# One content item, rendered with the site theme

GET /wps/wcm/connect/{library}/{siteArea}/{contentName}

# One content item, specific presentation template, no theme chrome

GET /wps/wcm/connect/{library}/{siteArea}/{contentName}?presentationtemplate={library}/{templateName}

# A single component (menu/list/personalization), rendered standalone

GET /wps/wcm/connect/{library}/{siteArea}?srv=cmpnt&source=library&cmpntname={library}/{componentName}

## Content — headless (JSON) APIs

# v1 — READ ONLY

GET /dx/api/core/v1/dxrest/webcontent/contents/{contentId} # anonymous

GET /dx/api/core/v1/mydxrest/webcontent/contents/{contentId} # authenticated

# v1 — search by Content Template

GET /dx/api/core/v1/{access\_type}/webcontent/search?type=Content&authoringTemplateID={ctId}

# v2 — READ + WRITE

GET /wps/mycontenthandler/wcmrest-v2/contents/{contentId}

POST /wps/mycontenthandler/wcmrest-v2/contents

PUT /wps/mycontenthandler/wcmrest-v2/contents/{contentId}

**Pre-flight checklist before any external integration works:** 1. Auth first — POST /dx/api/core/v1/auth/login (or use dxrest anonymous paths for public content). 2. Confirm the relevant **Virtual Resource** (e.g. WCM REST SERVICE) grants Editor access to the calling principal (Course 1, section C) — otherwise you'll get permission errors even with valid credentials. 3. Get real UUIDs (library/site area/content-template/content) from **Content Composer** → **hover** → **Copy UUID**, or from a GET response body — never guess them.